Cloud-Native Intelligent Hospital Operations Platform

DA1 Project Presentation

Cloud Computing – BCSE408L

Group Members:

Midhul Kiruthik M Madhan Karthikeyan Sachin VP Sharvesh

February 9, 2026

Contribution of Each Member

- Midhul Kiruthik M — System architecture, scheduling logic, forecasting design
- Madhan Karthikeyan — NLP pipeline and clinical summary processing
- Sachin VP — Backend services, Redis and Celery integration
- Sharvesh — Frontend dashboards and monitoring visualization

Abstract

Modern hospitals operate under high variability in patient arrivals and doctor availability. Existing cloud healthcare systems rely on static scheduling and batch analytics, leading to decision-making latency. This project proposes a cloud-native hospital operations platform that reduces latency using compute-driven scheduling, real-time workload forecasting, and asynchronous AI-based clinical summarization. The system adopts an event-driven architecture with distributed background workers, non-blocking APIs, and enforced observability, security, and disaster recovery guarantees.

1. Introduction

1.1 Topic & Context

Hospitals require operational systems capable of reacting in near-real time to cancellations, walk-ins, and fluctuating clinician workloads. Existing literature shows scheduling and forecasting are often decoupled from execution engines, while clinical documentation remains manual and blocks workflows.

1. Introduction (cont.)

1.2 Aim

Design and implement a cloud-native hospital operations platform that optimizes appointment scheduling, balances doctor workloads, and automates post-consultation documentation using distributed compute and real-time analytics.

1. Introduction (cont.)

1.3 Objectives

- Build a compute-driven Smart Appointment Load Balancer
- Implement real-time load forecasting with 30–60 s update cadence
- Provide asynchronous AI-based clinical summary generation
- Architect an event-driven, fault-tolerant system using background workers
- Enforce security, auditability, observability, and disaster recovery

1. Introduction (cont.)

1.4 Scope & Limitations

In Scope:

- OPD scheduling
- Short-horizon forecasting
- Asynchronous processing (Redis + Celery)
- Role-based access
- Local cloud simulation

Out of Scope:

- Automated diagnosis
- National EHR replacement
- Government health systems

1. Introduction (cont.)

1.5 Expected Outcomes

Reduced waiting times, balanced clinician workloads, predictive operational visibility, automated clinical documentation, and demonstration of scalable cloud-native design principles.

2. Research & Literature Synthesis

2.1 Annotated Citations (IEEE Style)

15 peer-reviewed research papers covering:

- Scheduling optimization
- Healthcare forecasting
- Streaming analytics
- Clinical NLP
- Event-driven architectures
- Cloud disaster recovery

2. Research & Literature Synthesis (cont.)

2.2 Literature Synthesis Matrix (Condensed)

- Dynamic scheduling lacks real-time execution
- Forecasting systems not operationally coupled
- NLP systems suffer from high latency
- Event-driven designs lack analytics depth
- DR studies not healthcare-integrated

2. Research & Literature Synthesis (cont.)

2.3 Problem Statement — Decision-Making Latency

Decision-making latency is the delay between a system state change and its operational response.

Root causes:

- Static scheduling
- Offline analytics
- Blocking workflows
- Manual documentation
- Weak observability and disaster recovery

No existing solution provides continuous state computation with automated action through asynchronous cloud workflows.

2.4 Key Features of the Project

- Compute-driven smart appointment scheduling using workload scoring
- Real-time workload forecasting with short update intervals
- Event-driven, asynchronous processing using Redis and Celery
- AI-based clinical summary generation outside the API critical path
- Built-in observability, fault tolerance, and disaster recovery guarantees

3. Technical Design Assets

3.1 High-Level System Architecture

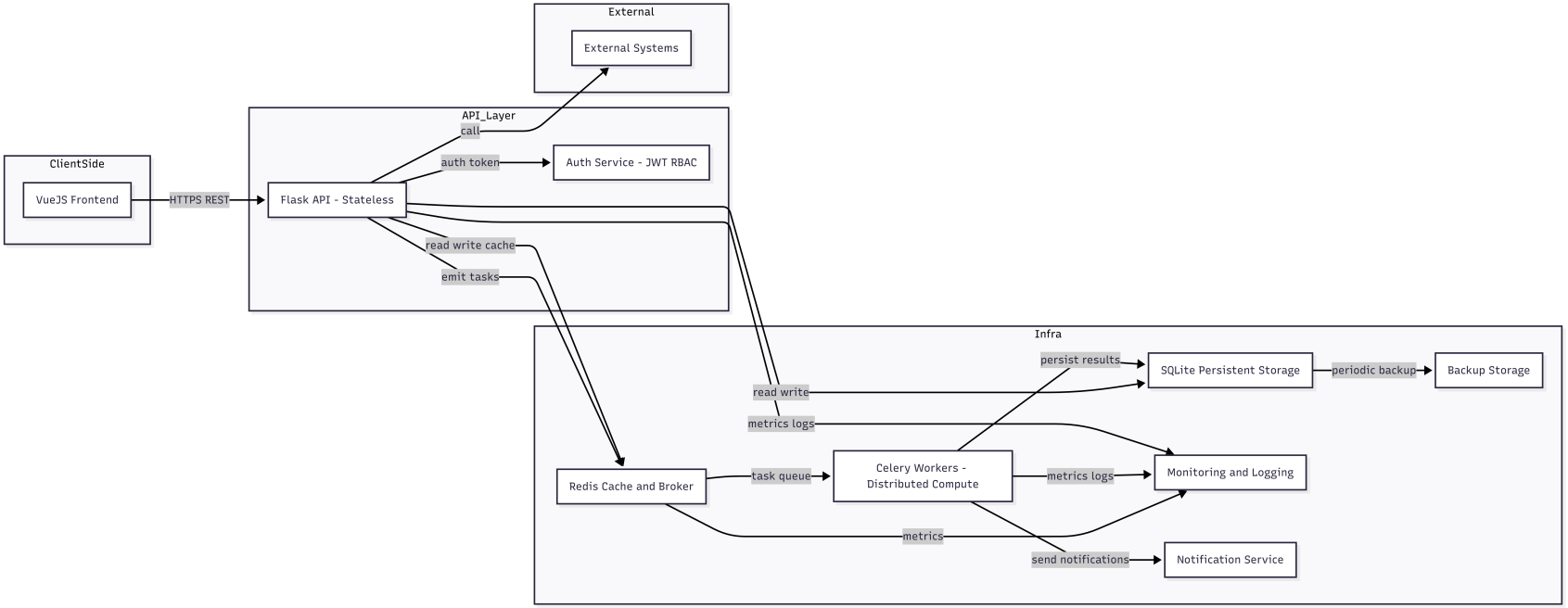

Figure: High-Level Cloud-Native System Architecture

Stateless APIs interact with Redis as cache and broker, Celery workers perform distributed computation, and persistent storage with monitoring ensures scalability and reliability.

3.2 Smart Appointment Load Balancer — Logical Flow

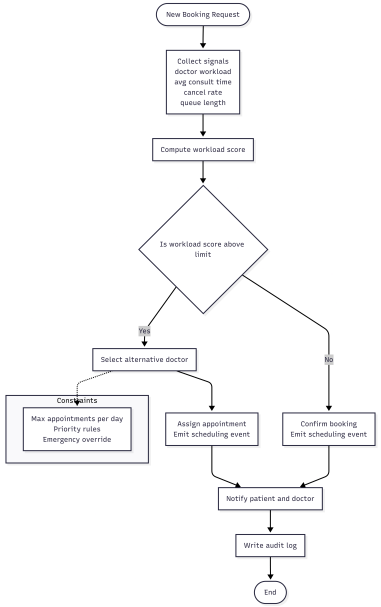

Figure: Smart Appointment Load Balancer Flow

Compute-driven appointment assignment using workload scoring, threshold checks, constraint enforcement, notifications, and audit logging.

3.3 Asynchronous AI Task Processing Pipeline

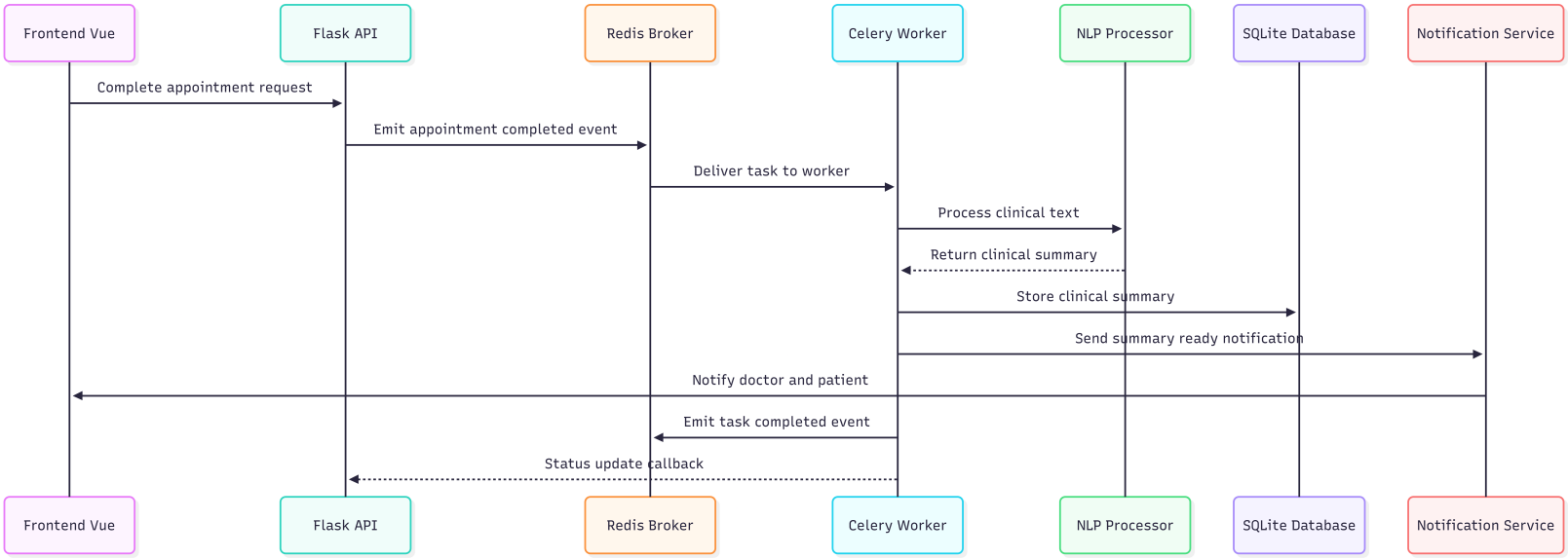

Figure: Asynchronous AI-Based Clinical Summary Generation

Appointment completion events trigger background NLP processing outside the API critical path.

3.4 Data Flow and Storage Design

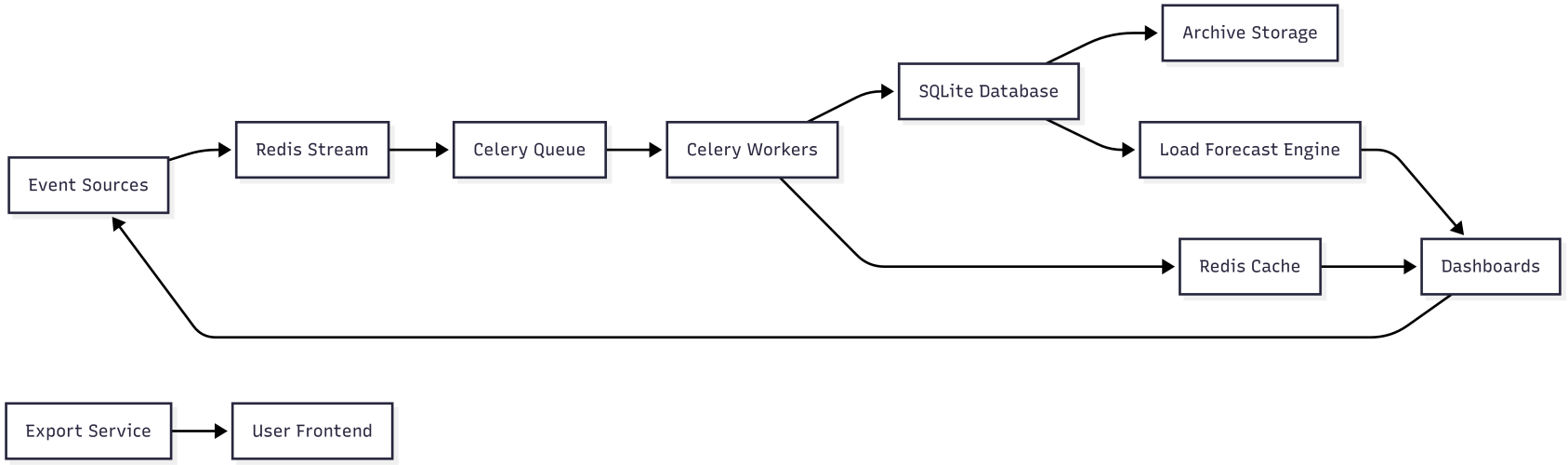

Figure: Event-Driven Data Flow and Storage Architecture

Redis streams and Celery queues drive data into cache, databases, analytics dashboards, and archival storage.

3.5 Reliability, Observability & Fault Tolerance

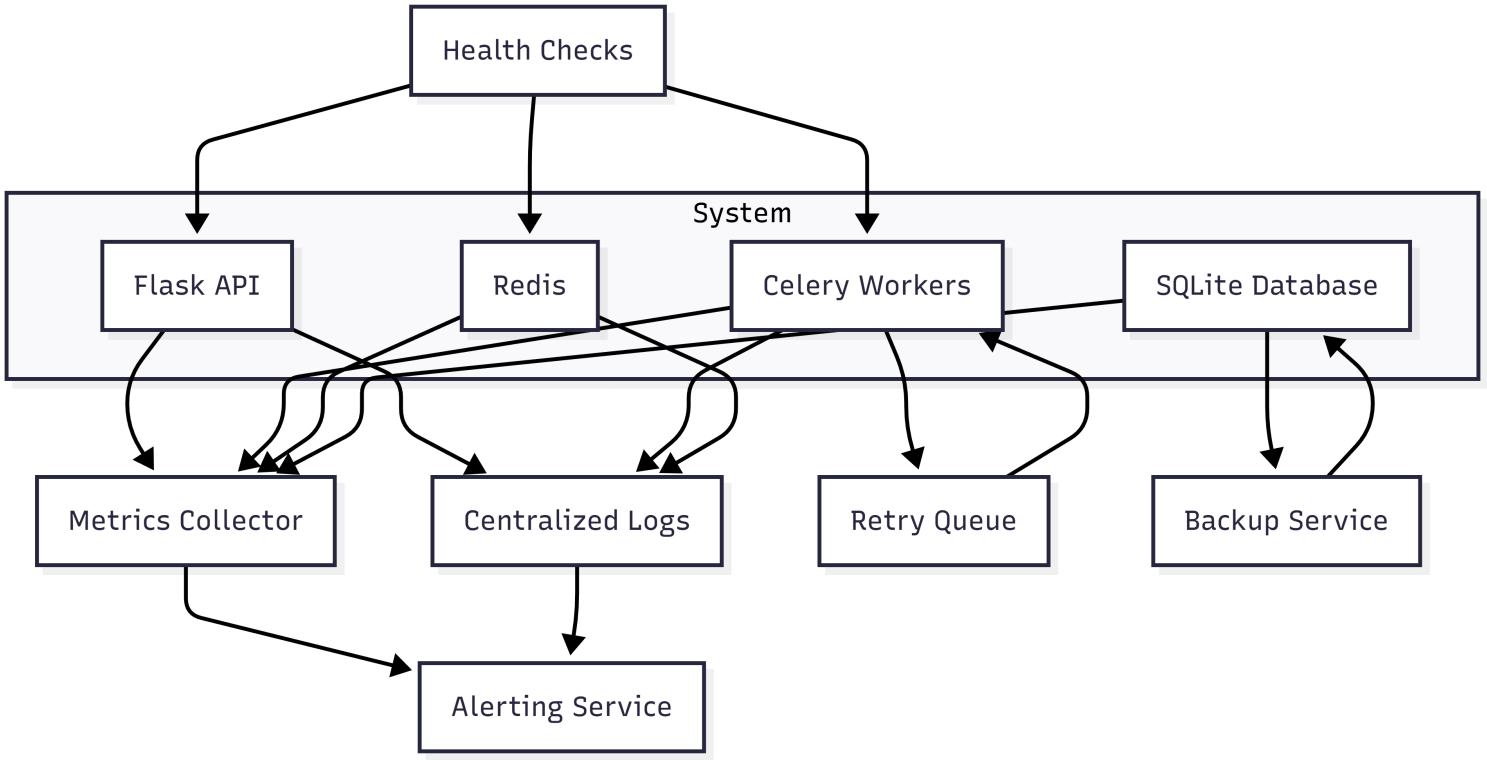

Figure: Reliability and Observability Architecture

System enforces Availability 99.9%, RPO 15 minutes, and RTO 30 minutes using monitoring, retries, alerts, and backups.

3.6 Security & Auditability Design

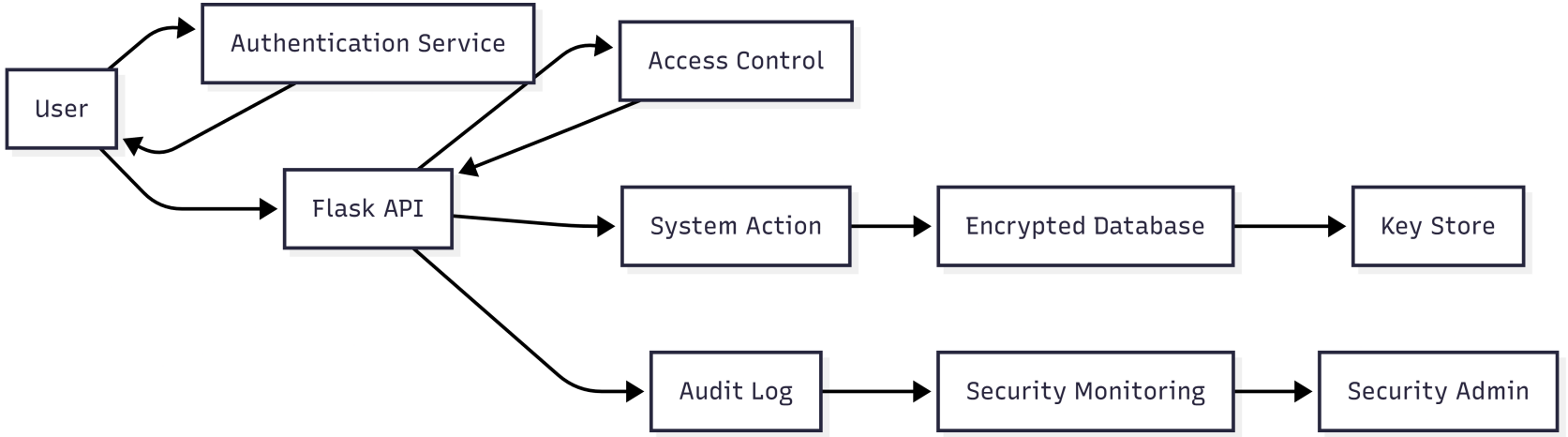

Figure: Security and Audit Flow

Authentication, RBAC enforcement, encrypted storage, immutable audit logs, and continuous security monitoring are enforced.

4. Data & Methodology

4.1 Data Dictionary

- **workload_score** — Composite metric representing doctor workload
- **wait_time_metric** — Predicted patient waiting time (minutes)
- **forecast_window** — Rolling time window for demand prediction
- **summary_text** — NLP-generated clinical summary
- **appointment_status** — Booked / Completed / Cancelled
- **patient_id** — Unique patient identifier

4. Data & Methodology (cont.)

4.2 Simulation Parameters

- Number of doctors: 30–50
- Patient arrivals per day: 400–600
- Arrival model: Poisson distribution
- Average consultation duration: 8–12 minutes
- Appointment cancellation rate: 10–15%

4. Data & Methodology (cont.)

4.3 Research Design

An iterative SDLC-driven research methodology was followed:

- Literature review and gap identification
- System architecture and design
- Incremental implementation
- Performance evaluation
- Design refinement based on results

5. Implementation Details

5.1 Technology Stack

- Frontend: Vue.js
- Backend: Flask (Python 3.11)
- Queue and Cache: Redis 6.x
- Background Workers: Celery 5.x
- Database: SQLite (prototype), PostgreSQL (production-ready)

5. Implementation Details (cont.)

- Forecasting models: Rolling regression, ARIMA, LightGBM
- NLP processing: Rule-based logic with DistilBERT / T5-small
- Monitoring and observability: Prometheus and Grafana

5. Implementation Details (cont.)

5.2 Security & Compliance

- JWT-based authentication and role-based access control
- TLS-encrypted communication channels
- Encrypted data storage at rest
- Append-only audit logging
- SIEM-ready security monitoring integration

6.2 Key Performance Indicators

- Scheduling decision latency (milliseconds)
- Reduction in patient waiting time (percentage)
- Successful overload reassignment rate
- NLP task completion time (seconds)
- System uptime compared against SLO targets

Appendix D — Annotated Literature Review (1/3)

- Zhuang et al., 2024 — Dynamic appointment scheduling using optimization techniques; lacks real-time execution coupling.
- He et al., 2025 — Robust optimization under uncertainty for hospital scheduling; no streaming integration.
- Seo et al., 2024 — Machine learning–based patient occupancy forecasting; detached from operational systems.
- King et al., 2025 — Streaming ML for real-time bed demand prediction; limited to bed management.
- Katiyar et al., 2023 — Event-driven microservice architectures in healthcare; minimal analytics depth.

Appendix D — Annotated Literature Review (2/3)

- Abacha et al., 2024 — Clinical NLP benchmark for error detection; not deployed in live systems.
- Oliveira et al., 2025 — Transformer-based clinical summarization; high inference latency.
- CBDR Survey, 2024 — Cloud disaster recovery metrics and best practices; lacks healthcare coupling.
- Zhang et al., 2023 — Queueing models for outpatient departments; static assumptions.
- Li et al., 2024 — Hospital workload prediction using time-series models; no automation layer.

Conclusion

This project demonstrated the design of a cloud-native hospital operations platform focused on reducing decision-making latency in appointment scheduling and clinical workflows.

By combining compute-driven scheduling, real-time workload forecasting, and asynchronous AI-based clinical summarization, the system improves operational efficiency while maintaining scalability, security, and fault tolerance.

The proposed architecture is cloud-agnostic, event-driven, and suitable for real-world hospital environments with high variability and strict reliability requirements.

Appendix D — Annotated Literature Review (3/3)

- Kumar et al., 2023 — Cloud-based hospital management systems; limited scalability.
- Rahman et al., 2024 — Secure healthcare data pipelines in cloud environments; no operational analytics.
- Chen et al., 2024 — Event streaming architectures using Redis and Kafka; not healthcare-specific.
- Wang et al., 2025 — AI-assisted medical documentation systems; blocking workflow design.
- Patel et al., 2024 — Observability frameworks for cloud-native systems; not domain-tailored.